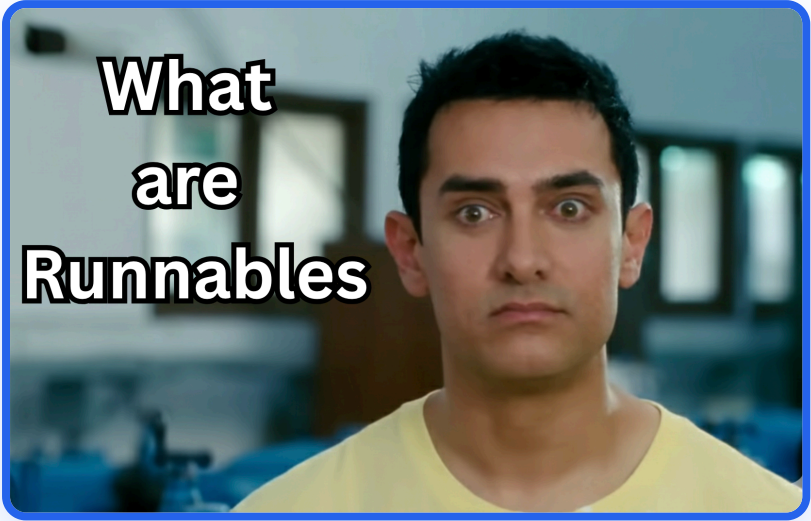


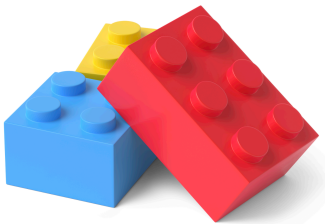
What are Runnables



Understand a **Complex** Concept in LangChain

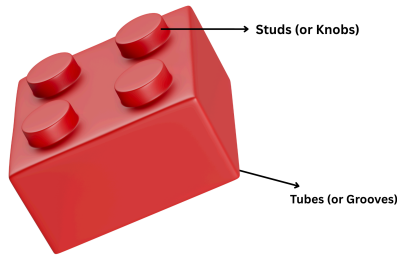
Runnables

with a Simple Lego Block Analogy



Naved Khan
Gen AI Engineer

+ Follow



Each Lego Block Has:

- An **Input Connector** (Tubes) – what it expects in
- An **Output Connector** (Studs) – what it produces

The Magic:

Since all blocks share the same connector style, you can **snap them together** using the

| operator!

Examples of "Lego Blocks" in LangChain:

PromptTemplate

dict → string

LLM / ChatModel

string → text

Parser

text → dict/JSON

RunnableLambda

any → any

RunnableBranch

value → conditional

RunnableParallel

dict → multi-output



The Problem (Before Runnables)

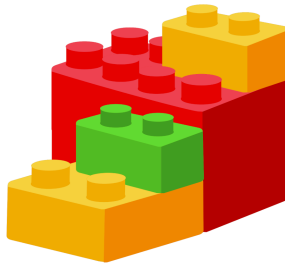
In normal Python:

- A **prompt** is a function
- An **LLM call** is another function
- A **parser** is another function

But you can't easily **pipe them together** in one consistent way!

LangChain's Solution

"Let's make **everything a Runnable**, so they can all be chained together the same way."



prompt

| llm

| parser

THE RESULT

A **universal adapter** – whether it's a prompt, a model, a parser, or your own function, once it's a Runnable, you can plug it into any chain!



How to Recognize a Runnable?

3 / 9

Any object in LangChain that implements these **3 methods** is a Runnable:

.invoke(input)

Run **once** with a single input

.batch([inputs])

Run in **batch** mode with multiple inputs

.stream(input)

Stream outputs as they're generated

That's why you can chain them:

`prompt` is a Runnable (PromptTemplate)

`llm` is a Runnable (ChatGoogleGenerativeAI)

`parser` is a Runnable (StrOutputParser)

KEY DEFINITION

Runnable = a callable object with a standardized interface that allows it to be composed inside LangChain workflows.



Naved Khan
Gen AI Engineer

3 / 9

Example 1: PromptTemplate Runnable

4 / 9

Input: dictionary → **Output:** formatted string

```
from langchain_core.prompts import PromptTemplate

# Create a prompt template
prompt = PromptTemplate.from_template("Say hi to {name}")

# Invoke it with input
result = prompt.invoke({"name": "Naved"})

# output: "Say hi to Naved"
```

Another example with input variables:

```
report_prompt = PromptTemplate(
    template = 'Generate a report on {topic}',
    input_variables = ['topic']
)

report_prompt.invoke({"topic": "Generative AI"})
# output: "Generate a report on Generative AI"
```

REMEMBER

PromptTemplate takes a **dict** and outputs a **formatted string**!





ChatModel / LLM Runnable

Input: string/message → **Output:** AI response

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    api_key=gemini_key
)

llm.invoke("Hello, who are you?")
# output: "I am an AI assistant..."
```



OutputParser Runnable

Input: AI message → **Output:** parsed string/object

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

result = parser.invoke("Here's your summary")
# Converts AI message to plain string
```



Convert Any Function to a Runnable

6 / 9

Use **RunnableLambda** to turn ANY Python function into a Runnable!

Step 1: Your normal Python function

```
# A regular Python function
def square(n):
    return n * n

print(square(2)) # output: 4
```

Step 2: Convert to Runnable

```
from langchain_core.runnables import RunnableLambda

# Convert function to Runnable
sq = RunnableLambda(square)

# Now use it like any other Runnable!
sq.invoke(5) # output: 25

# Can also batch!
sq.batch([2, 3, 4]) # output: [4, 9, 16]
```

POWER TIP

RunnableLambda unlocks **custom logic** inside your chains!



Naved Khan
Gen AI Engineer

6 / 9

Because **all are Runnable**, you can connect them with the  operator!

```
from langchain_core.prompts import PromptTemplate
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.output_parsers import StrOutputParser

# Create individual runnables
prompt = PromptTemplate.from_template("Say hi to {name}")
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
parser = StrOutputParser()

# Chain them together!
chain = prompt | llm | parser

# Run the entire chain
result = chain.invoke({"name": "Naved"})
```

prompt



llm

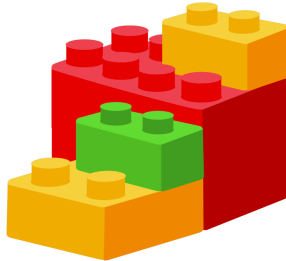


parser

The entire chain is ALSO a Runnable!

You can `invoke`, `batch`, or `stream` the whole thing.





A Runnable is like a Universal Adapter

It doesn't matter if it's a prompt, a model, a parser, or even your own function — once it's a Runnable, you can plug it into any chain.

✓ **RunnableLambda** – Custom functions in chains

✓ **RunnableBranch** – Conditional routing

✓ **RunnableParallel** – Parallel execution

REMEMBER

Runnable = callable object with **invoke**, **batch**, **stream** that can be composed in LangChain workflows.



Thanks for Reading!



Naved Khan

Gen AI Engineer

Save this for later
Share with someone learning LangChain!

